Search

Engineering

Updated Sep 01 Edit Share

Scriptable Collector (scol) sources

By Janmejay Singh 9 min 11 Add a reaction

Scol-sources are active (polling) sources (in dataplane) that can pull data from 3rd party APIs. At a high level they have following properties:

- Are HTTP clients (in a future version this may be extensible to other protocols)
- Interval driven (eg. poll once in 30 seconds)
- With the help of script (provided as part of config) can navigate arbitrarily complex data-model. Eg. can deal with several paging strategies in the response where each page of response has several entries and each entry is a url that needs to be fetched independently and emitted as an event.
- Are governed by a single dataplane process but fetching and event-generation can scale to a large dataplane deployment.
- The scripting capability is core this source-type. Each source has a single `Lua` fragment that powers data-model navigation, url-construction, pagination strategy etc.
- Script can create checkpoints at arbitrary granularity and use these to upper-bound regression upon crash / upgrade etc.

User / product interface

Users do not configure these generic sources directly, IOW users do not write Lua scripts that are at the heart of this.

Manager ships with support for some scol-source types such as M365, Dynatrace etc and the data-model specific `Lua` scriplets for each source-type ship with Manager. Users' responsibility is limited to configuring top-level-URL, Auth, polling-interval etc.

Because these are 3rd party APIs and there are limits to the degree to which we can test support for these sources. Ability to fix bugs in data-model navigation, URL construction etc in the field is key to good customer-experience. Manager would allow overrides to scriplets at either Site or Source level so if we discover bugs in the field (or differences in endpoint behavior / data-model due to differing service-versions) we can use an override to fix stock scriplet.

Design

The usage is best described thru a hypothetical example.

Scol (dataplane) config

A typical scol-config may look something like (this is not a completely accurate representation of config, this was built as a placeholder for ideas before SCol was built, the real thing differs in several places, but matches the description here in terms of actual behavior).

```
100000000000001557_source:
    rpc_domain:
        foo.com:
            retry:
                max_retries: 3
                initial_backoff: 2s
                max_backoff: 30s
                jitter: 2s
            auth:
                strategy: oauth2
                client_id: <client_id>
                client_secret: <client_secret>
                token_url: <token url>
            tls:
                ca_file: <path>
```

```
                cert_file: <path>
                key_file: <path>
```

Updated Sep 01 Edit

```
                server_name: foo.com
                verify_certificate: true
                verify_hostname: true
            timeout: 20s
        test.foo.com:
            auth:
                strategy: bearer
                token: <token>
            tls:
                enabled: true
            timeout: 10s
        bar.com:
            auth:
                strategy: basic
                user: <user>
                password: <pass>
            tls:
                enabled: true
            timeout: 20s
    seed_checkpoints:
        foo.com/discovery: "MjAyNS0wNS0zMFQyMDoxNzozNVo="
        bar.com/fetch: "MjAyNS0wNS0zMFQyMDoxNzozNVo="
    trigger:
        interval: 30m
        await_completion: false
    config:
        endpoint: <url>
        itvl_seconds: 30
        publisher_id: <...>
        tenant_id: <...>
        page_sz: 100
    scale_out:
        port: 10005
        runtimes: 2
    start_routine: start
    script: |
        json = require("json")

        function update_discover_checkpoint(config, result, ctx)
            if result.discover_fetch.discover_succeeded then
                error("update_discover_checkpoint: unexpected result: " .. tostring(result.discover_fetch))
            end
            set_ckkpt("foo.com/discovery", ctx.new_cp)
        end

        function update_collect_checkpoint(config, result, ctx)
            local success = 0
            local total = 0
            local highest_time_str = ""
            for _, r in ipairs(result) do
                if r.success then
                    success = success + 1
                end
                total = total + 1
                if r.time and r.time > highest_time_str then
                    highest_time_str = r.time
                end
```

```
            end
            if (success / total) > 0.9 then
```

Updated Sep 01 Edit

```
        end

        function collect(config, res, ctx)
            if res.status == 200 then
                local data = json.decode(res.body)
                if data and data.events then
                    local results = {}
                    for i, e in ipairs(data.events) do
                        e.referrer = ctx.referrer
                        assert(emit(e))
                    end
                end
                return {success = true}
            end
        end

        function discover(config, res)
            local cp = get_chkpt("bar.com/fetch")
            if res.status == 200 then
                local data = json.decode(res.body)
                local results = {}
                if data and data.entries then
                    for i, e in ipairs(data.entries) do
                        results[i] = assert(
                            fetch{
                                params = {
                                    url = e.url, -- may be a test.foo.com or bar.com url
                                    headers = {
                                        ["Accept"] = "application/json",
                                        ["StartValue"] = cp},
                                    method = "GET"},
                                context = {referrer = res.url},
                                fn = "collect"},
                            "fetching " .. e.url .. " failed")
                    end
                end
                mark_complete{discover_succeeded = true}
                on_complete{
                    result = results, -- must have: serializable keys, results for values
                    fn = "update_collect_checkpoint"}
            else
                error("discover failed: unexpected status code: " .. tostring(res.status))
            end
        end
        -- 2006-01-02T15:04:05Z07:00"
        tm_fmt = "%Y-%m-%dT%H:%M:%SZ%z"

        function start(config)
            local headers = {["Accept"] = "application/json"}
            local start_tm_str = get_chkpt("foo.com/discovery")
            local start_tm = assert(strptime(start_tm_str, tm_fmt), "couldn't parse start_field")
            local end_tm = start_tm + (config.itvl_mins * 60)
            local end_tm_str = assert(strftime(end_tm, tm_fmt), "couldn't format end_tm")
            local query = {
                ["contentType"] = "Audit.AzureActiveDirectory",
                ["start_field"] = start_tm_str,
                ["end_field"] = end_tm_str,
```

```
                ["PublisherIdentifier"] = config.publisher_id}
            local q_result = assert(
```

Updated Sep 01 Edit

```
                        url = config.endpoint,
                        headers = headers,
                        query = query,
                        method = "GET"},
                    fn = "discover"},
                "discovery failed")
            assert(on_complete{
                result = {discover_fetch = q_result},
                context = {new_cp = end_tm_str}, -- must have: serializable keys and values
                fn = "update_discover_checkpoint"})
        end

    lang: lua

```

Params

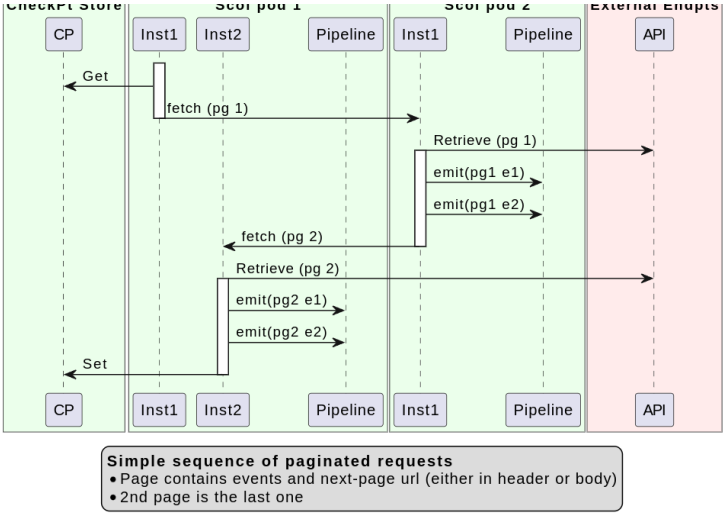
- **rpc_domain** captures auth config and other non-functional properties by hostname. A complex collector may need to talk to multiple domains and may want different timeout and retry behavior for each one.
- **seed_checkpoints** injects values for checkpoints (identified by opaque string keys) to be used the first time, subsequent runs do not use this value
- **trigger** defines interval of collection and whether to start collection even if the previous collection hasn't finished
- **config** is a free-form map and is passed to the script, it is not interpreted by the collector
- **scale_out** is an optional field and configures port to distribute work across a larger set of dataplane nodes and number of script-runtimes to maintain per node
- **start_routine** identifies a function-name (in the script) to be called to initiate collection
- **script** defines the collection algorithm
- **lang** specifies the language to be used to interpret the script, in the first cut Lua is the only supported language

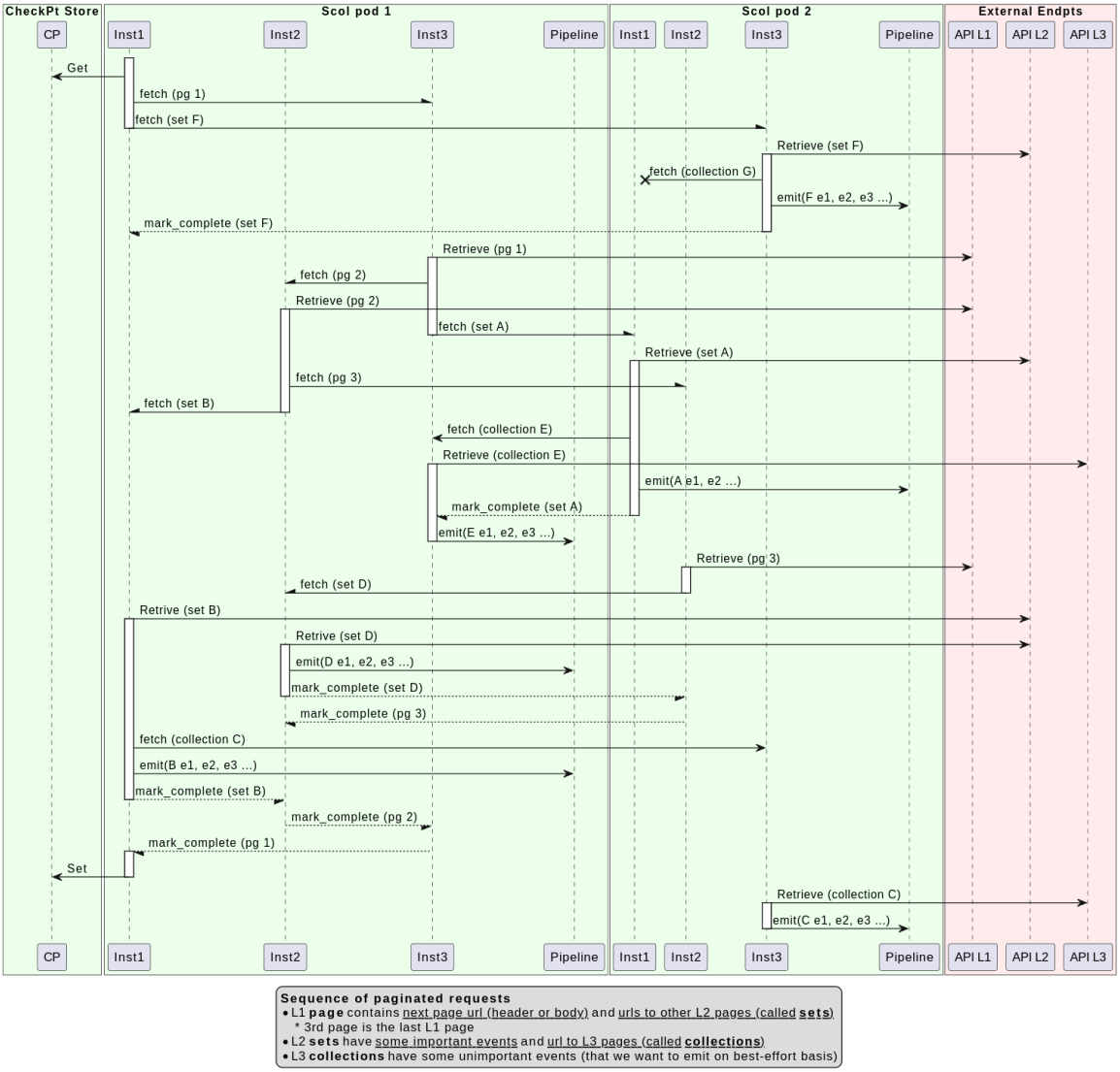
Lua APIs

- **fetch** accepts a table with key `params` that are used to make the HTTP request and `fn` which is name of the function to be called with the response received. It also includes an optional `context` attribute that carries additional metadata from work enqueue to processing
  - `fetch` returns an identifier (`work-identifier` , which is a uuid) to the fetch work enqueued
  - `fn` is called with response which includes headers and body of the HTTP response
- **exec** accepts a table with key `params` that control subprocess input, environment and cmdline args etc, works in a way that is very similar to `fetch`
  - Is disabled if `sandbox` is not enabled, `exec` is disabled to avoid security problems.
- **on_complete** allows the caller to attach a hook to process the outcome (fetch and processing) of `fetch` s enqueued. Param `result` identifies units of work to await in form of a table with any serializable type for keys (number, string) and `work-identifier` s for value.
  - a `fetch` handler (`fn`) can either explicitly mark the work complete (use `mark_complete`)
  - OR completion of the `on_complete` hook in work issued would lead to `on_complete` `fn` call
  - OR when neither is done, work is considered complete as soon as fetch returns (which is identical to calling `mark_complete` as last thing in the body of `fn`)
  - The diagram below shows various scenarios related to completion
- **mark_complete** allows script to mark the executing collection work complete at an arbitrary point
- **get/set_chkpt** either get or set value of a given checkpoint key
- some common data-time / string manipulation functions such as `strptime` and `strfime` are made available by `scol` script-env
- `json` serde is supported

- **emit** helps collector emit events (and accepts event-type which defaults to `log`)

Updated Sep 01 Edit





`Inst1` , `Inst2` ... etc (in the diagram) represent Lua runtime instances. White-rectangles represents running script (Lua code) while arrows outside the rectangles represent IO being managed by scol source with no assistance from Lua script.

Non-functional properties

- Lua runtime is only used for compute, IO is always performed outside runtime. Neither `fetch` nor `on_complete` block for the fetch-

Updated Sep 01 Edit

- `fetch` work may be distributed across several dataplane processes (although in v1 it'll be executed locally on the same dataplane, possibly in a different runtime instance)
- All tables exchanged as part of `fetch`, `on_complete`, `mark_complete` etc must only contain serializable types (number, strings, tables, bools, but no functions, userdata etc)
- Work distribution (and completion-wait) is achieved thru messages exchanged between dataplane instances, but this would not be built as part of v1.
- Authentication (token refresh) etc are handled transparently by `scol` (behind `fetch` call)

RPCs (not part of v1)

- A message is sent to the chosen port of the dataplane service when `fetch` work is issued, this contains the IP-address and listening-port of the sending node that receives the completion notifications.
- We may eventually add additional retry-config to allow these messages to be used reliably.
- Dataplane `protobuf Log` event-array may be used with (internal, 1-per-collector-source) socket-source for `fetch` RPCs. Attributes `fields` or `value` (with `ValueMap`) can be used to pack details such as type of event (`fetch` or `mark_complete`, the `fetch` event's origin (which like `arp` is used to send `mark_complete` notification, which powers `on_complete` when we support scale-out).
- Scale out (when built) will be deployed transparently upon site-upgrade, however the port will need to be assigned, so users will likely need to make a source-update (assigning port).

Deployment in Site

- **v1**
  - Dataplane pod replaces observo-collector and control-agent is enhanced to re-use `PatchConfigMap` for collector config too. The race-prone impl powering observo-collector config distribution is deleted.
  - There is a single dataplane pod, scale-out is not supported, however multiple runtimes are used and this will allow significant amount of vertical scaling
  - Checkpoints are stored in a PV in v1 in 1-value per file arrangement where files are organized by check-pt name under directories by source-id.
- **future improvements**
  - **Single dataplane deployment** would allow us to avoid a separate dataplane deployment and two different categories of dataplane config etc. In this case dataplane level (`trigger` based) scheduling will be dropped in favor of a job-queue to which control-agent enqueues top-level jobs, allowing one of the dataplanes to pick the top-level job (after which the regular scale-out will be used, avoiding job-queue altogether).
    - **Use of job-queue for scaling** `fetch` doesn't appear to have any technical advantages, it just complicates the design subjecting queue to additional stress, so we avoid this.
    - **Near-saturation sites** (such as Informatica) would likely be unfit for this design, so we may avoid this altogether (forever maintaining two separate dataplane deployments for cost / operational reasons).
  - **Scale-out** to multiple pods
    - **Use job-queue** only for triggering the top-level collection-cycle, this can be done by control-agent.
  - **Support for additional scripting languages** appears to have no additional value either, although this is something that will be possible (the current design doesn't use any Lua specific features like `coroutines` etc). Due to high performance, Lua is likely to remain the preferred language.
  - **DB based checkpoints** may be used in the future version (to allow scaling beyond a single pod). We will need to introduce DB to the site for Iceberg catalog so this doesn't impose any additional complexity in terms of number of stateful systems in site.

Changes in Manager

Each source-type (say M365, Salesforce etc) will have their corresponding `script` s shipped as part of site-manager binary (using `embed.FS`) and will be used except when overridden. This will be a map keyed by `SourceTemplate_SourceType` to `script` string.

Overrides would be supported at `org-id` level and would go into the table `collector_script_overrides` (keyed by `org_id` and `source_type`).

Updated Sep 01 Edit

collector_script_overrides (pri key: <org_id, source_type, version>)	
org_id	string
source_type	int (`SourceTemplate_SourceType`)
version	bigint
script	string
created_at	timestamp with time zone
updated_at	timestamp with time zone
created_by	string
updated_by	string
status	string (enum ACTIVE / DELETED)

We shouldn't really need overrides at source-type and site-level because the org-override can use branches to deal with situations like a mix or old and new versions of the same service.

The desired usage for this is for us to get on a call and setup overrides for users (or in demo env etc) when they hit a problem. These overrides would eventually flow back to the master-copy (in site-manager) and be shipped as part of next release fixing the original bug. Users would want to delete their overrides after upgrading all their sites to continue to benefit from improvements in the scripts we ship with.

The UX for this is TBD but the page to list and controls to modify / delete override would likely make sense in the settings space.

We'll migrate all existing collector types over to dataplane-scol(s) within a single release, however backwards compatibility will need to be maintained until sites upgrade, so current otel-collector config-mapping and related APIs / code in manager will need to be retained until one or more releases after the initial one.

Implementation plan

- Copy over http-client source in `vector` and strip away unwanted parts
- Introduce transparent token-refresh (similar to otel-collector), we'll have a hook that is called that gets to push additional headers on the requests
- Implement a 3rd-party-service test-double that allows us to declaratively setup HTTP endpoints with desired payloads
- Use this to test with multiple Lua runtimes
- Implement manager support once source is supported in `vector`

Open questions

- Do we need to share auth-details across dataplane instances (or Lua runtimes)? Or would several tokens issued on the same client / secret etc work at the same time, across processes?
- UI / UX for exposing overrides.

Related content

Observo Agent Design
Engineering

Pipelines
Sales Engineering

Observo Competition
Sales Engineering

Updated Sep 01 Edit

Design Specs & Proposals
Engineering
More like this

Site Manager
Engineering
More like this

Worker-group proposal
Engineering
More like this

Add a comment Add labels Add a reaction